

life_expectancy_analysis

April 9, 2025

1 Index: Life Expectancy and GDP Per Capita Analysis

1.1 1. Introduction

- Background on life expectancy and GDP per capita.
- Importance of studying this relationship.
- Clearly state your hypothesis.
- Overview of the datasets used.
- Structure of the report.

1.2 2. Data Preparation and Cleaning (`df_tidying`)

- **2.1 Data Import, Cleaning, and Merging**
 - **2.1.1 Data import**
 1. Issues with initial dataset
 2. Importing correct data from official sources
 3. Common data transformations
 - **2.1.2 Updating Template**
 1. Renaming columns to be more user friendly throughout project.
 2. Adjusting country data type to be string
 3. Removing additional data entries in dataset.
 - **2.1.3 Data Cleaning and Merging**
 1. World Bank Datasets
 2. WHO Datasets
 3. Our World in Data (OWID) Datasets
- **2.2 Filtering and Feature Selection**
 1. Retained variables (*country, year, life_expectancy, GDP, etc.*).
 2. Converting variables to correct data types (*string, integer, float, etc*)
 3. Rounding float values to 2nd decimal
- **2.3 Enriching the Data**
 1. Adding regional and sub-regional classifications.

- 2. Assigning income classes based on World Bank classifications.
- **2.4 Handling Missing Data**
 - Methods: extrapolation, linear regression, polynomial regression, and imputation.
 - Justification for each approach.
- **2.5 Data Type Adjustments**
 - Ensuring numerical and categorical variables are in correct formats.
 - Display dataset information
- **2.6 Exporting Cleaned Dataset to CSV**
 - Cleaned Dataset is exported to CSV file for analysis

1.3 3. Data Wrangling and Statistical Analysis (`df_wrangling`)

- **3.1 Summary Statistics for All Variables**
 - Mean, max, min, median, standard deviation, interquartile range (IQR).
 - Identifying key trends in life expectancy and GDP.
- **3.2 Preparing Data for EDA**
 - DataFrames for GDP per capita and life expectancy by income class.
 - Boxplots comparing life expectancy of bottom vs. top GDP countries.

1.4 4. Exploratory Data Analysis (EDA) (`descriptive_analysis` & EDA)

- **4.1 Correlation and Heatmap Analysis**
 - Correlation matrices for the complete dataset and by region.
 - Heatmaps highlighting relationships between variables.
- **4.2 Scatterplot Analysis**
 - Life expectancy vs. all other variables.
- **4.3 Descriptive Statistics**
 - Summary of life expectancy and GDP per capita (by region and income class).
- **4.4 Distribution Analysis**
 - Histograms for life expectancy and GDP per capita (overall and by region).
 - Distributions by income class.
- **4.5 Line Graphs Over Time**
 - Trends of life expectancy and GDP per capita over time.
- **4.6 Boxplots for Comparison**
 - Life expectancy and GDP per capita across different regions and income classes.
- **4.7 Deep Dive into GDP and Life Expectancy**
 - Summary statistics by income class.
 - Heatmaps by income class.
 - Scatterplots by income class.

- Boxplots comparing bottom vs. top GDP countries.

1.5 5. Progression Toward the Final Visualization

- **5.1 Key Insights from Exploratory Data Analysis**
 - Identification of significant trends and patterns.
 - Statistical evidence and visual representations supporting the observed relationships.
 - Consideration of outliers and their implications.

1.6 6. Final Visualization: Comprehensive Analysis and Interpretation

- **6.1 Data-Driven Insights Across Income Classes**
 - Interpretation of trends within different income categories.
 - Key findings derived from the final visualization.
- **6.2 Validating the Hypothesis**
 - A structured explanation of how the final visualization substantiates the proposed relationship.

1.7 7. Conclusion and Discussion

- **7.1 Summary of Findings**
 - Recap of evidence supporting the hypothesis.
- **7.2 Policy Implications**
 - Potential applications for health and economic policy.
- **7.3 Limitations and Future Research**
 - Limitations of the dataset and methodology.
 - Areas for further investigation.

2 2. Data Preperation and Cleaning

2.1 2.1 Data Import, Cleaning, and Merging

2.1.1 2.1.1 Data Import

Issues with the Initial Dataset:

The initial dataset, sourced from Kaggle (uploaded by user KumarRajarshi), was found to contain incorrect data. Upon reviewing the comments from other users and conducting independent verification against official sources such as the World Bank, significant inconsistencies were identified. The population figures did not align with World Bank records, and further investigation revealed that all variables contained erroneous or unreliable data. Due to incorrect uploads or misused sources, the dataset was deemed unsuitable for analysis.

Importing Correct Data from Official Sources:

To ensure data accuracy, the necessary datasets were imported from three authoritative sources: - **World Health Organization (WHO)** - **World Bank (WB)** - **Our World in Data (OWID)**

2.1 WHO Datasets The following CSV files were downloaded from the WHO database and stored under `Datasets/Raw_datasets/WHO`: - **Adult Mortality Rate** - **Child Deaths per 1000 Births** - **Infant Mortality** - **Under-5 Mortality**

These datasets were read using Pandas' `.read_csv()` function and stored as variables. #####
2.1.1 Adult Mortality Dataset Processing - Selected relevant columns: 'Location', 'Period', 'Dim1', 'FactValueNumeric' - Renamed columns: - 'Location' → 'country' - 'Period' → 'year' - 'Dim1' → 'sex_classification' - 'FactValueNumeric' → 'adult_mortality' - Standardized data types (`.astype('string')`) - Standardized country names to match the project template using `.replace()`: - Cote d'Ivoire → Côte d'Ivoire - Netherlands (Kingdom of the) → Netherlands - North Macedonia → The former Yugoslav Republic of Macedonia - Türkiye → Turkey - Filtered the dataset to include only: - Data from 2000 to 2015 - Entries labeled as "Both sexes" - Dropped the `sex_classification` column

2.1.2 Child Deaths, Infant Mortality, and Under-5 Mortality Processing

- Selected relevant columns: 'Location', 'Period', 'Dim1', 'Dim2', 'Value'
- Renamed columns:
 - 'Location' → 'country'
 - 'Period' → 'year'
 - 'Dim1' → 'age_classification'
 - 'Dim2' → 'cause'
 - 'Value' → 'deaths'
- Filtered and pivoted the datasets to obtain structured tables for analysis.

2.2 World Bank Datasets The following CSV files were downloaded from the World Bank database and stored under `Datasets/Raw_datasets/World_Bank`: - **Population** - **GDP per Capita** - **GNI per Capita** - **Health Expenditure Percentage** - **Life Expectancy** - **Education Expenditure** - **Unemployment Percentage** - **Alcohol Consumption** - **Government Health Expenditure**

These datasets were read using Pandas' `.read_csv()` function and stored as variables: - `world_population` (loaded with `on_bad_lines='skip'` for data integrity) - `wb_gdp` (GDP per capita) - `wb_gni` (GNI per capita) - `wb_health_exp_perc` (Health expenditure % of GDP) - `world_life_expectancy` (Life expectancy at birth) - `world_education_expenditure` (Education expenditure) - `wb_world_unemployment_total` (Unemployment %) - `world_alcohol_consumption` (Alcohol consumption per capita) - `world_total_expenditure_gov` (Government health expenditure)

2.3 Our World in Data (OWID) Datasets The following CSV files were downloaded from OWID and stored under `Datasets/Raw_datasets/Our_World_in_Data`: - **BMI (Male and Female)**

These datasets were read and stored as: - `world_in_data_bmi` (Male BMI) - `world_in_data_bmi_female` (Female BMI)

3. Common Data Transformations To standardize the imported datasets, the following transformations were applied across all datasets: - **Column Renaming:** Standardized column names to match project conventions. - **Year Filtering:** Restricted data to the 2000–2015 period. - **Country Name Standardization:** Aligned country names with the project template. - **Data Type Conversion:** Ensured categorical variables were stored as strings. - **Sorting & Indexing:** Sorted by country and year for consistency.

2.1.2 2.1.2 Updating Template

To ensure consistency and usability throughout the project, the dataset template was refined through column renaming, data type standardization, and the removal of incomplete country records. These modifications enhanced the dataset’s structure, making it more accessible for analysis.

Renaming Columns for Readability:

The dataset’s original column names contained inconsistent capitalization, spaces, and special characters, which could hinder efficient data manipulation. To improve clarity and maintain a uniform naming convention, all column names were transformed to lowercase with words connected by underscores.

Methodology:

- The `.columns` method was used to inspect the dataset’s variable names.
- The `.rename(columns={})` method was applied to systematically rename variables.
- **Example transformation:**

```
life_expectancy.rename(columns={'Country': 'country', 'Year': 'year'},
inplace=True)
```
- This change ensured that variable names were standardized for consistency in all future operations.

Standardizing Country Data Type

The country variable was initially stored as an object type with inconsistent formatting. To maintain uniformity across datasets, it was explicitly converted to a string format.

- Transformation applied:

```
life_expectancy['country'] = life_expectancy['country'].astype('string')
```
- This standardization facilitated smooth dataset merging and prevented potential mismatches during analysis.

Removing Incomplete Country Entries

Some countries in the dataset had records for only a limited number of years, leading to inconsistencies in time series analysis. These incomplete entries were identified and removed through the following process:

1. The `.value_counts()` method was used to determine the frequency of each country in the dataset.
2. The result was converted into a dictionary where country names served as keys and their respective entry counts as values.
3. A list comprehension was applied to extract only countries with more than one entry:

```
valid_countries = np.array([key for key, value in unique_countries.items()
if value > 1])
```
4. The length of the resulting array was printed using the `len()` function to verify the number of valid countries.
5. A lambda function was used to scan through the dataset, retaining only entries present in `valid_countries`, while the rest were replaced with NaN values.
6. Finally, rows containing NaN values in the country column were dropped using the `.dropna(subset=['country'])` method to ensure that only countries with complete records remained.

2.1.3 2.1.3 Data Cleaning and Merging

This section outlines the data cleaning and merging processes for the three main data sources: World Bank, WHO, and Our World in Data. The objective was to ensure consistency across datasets, resolve naming discrepancies, and integrate the data into a unified structure.

1. World Bank Datasets Selecting Relevant Data

- Data from the World Bank was filtered to include only years 2000 to 2015.
- A list of countries from the World Bank dataset (`country_list_wb`) and a corresponding list from the life expectancy template (`country_list_le`) were created to identify common and mismatched country names.

Country Name Alignment

- Countries with mismatched names were identified by comparing `country_list_le` to `country_list_wb`.
- The `extractOne` function from the `thefuzz` library was used to match similar country names, with a score cutoff of 80.
- Manual adjustments were made for unmatched countries (e.g., “Korea, Dem. People’s Rep.”).
- The corrected names were applied to a cleaned version of the World Bank dataset (`world_pop_cleaned`).

Cross-Checking Countries

- A cross-checking list identified countries present in the life expectancy template but missing in the World Bank dataset (`countries_cross_check`).
- Missing countries such as Dominican Republic, Kyrgyzstan, and Swaziland were dropped from the Template.

Creating a Unified Dataset

- A new template (life_exp_clean) was created and sorted by country and year.
- The function modified_wb_pop_by_year() was developed to extract yearly data for population, GDP, health expenditure, and other variables.
- The cleaned and formatted World Bank dataset (wb_pop_sorted) was used to update life_exp_clean.
- This process was repeated for population, GDP, GNI, health expenditure, government health expenditure, life expectancy, unemployment, education expenditure, and alcohol consumption.

2. WHO Datasets This section covers how data from the World Health Organization (WHO) was cleaned and merged into the main dataset. The WHO data primarily contributed to mortality statistics, including adult mortality, infant mortality, and under-5 mortality. WHO country names matched those in life_exp_clean, requiring no renaming.

2.1 Adult Mortality

- Countries absent from WHO data were identified and stored in countries_not_included.
- The adult_mortality column was updated using a left join with WHO data.
- Any missing values in adult_mortality were handled using the where() function, ensuring consistency.

2.2 Infant Mortality

- The dataset included mortality data for different causes of infant deaths. To standardize this, a new column, infant_deaths, was created by summing all causes of infant mortality for each country-year combination.
- This was done using a lambda function to apply the sum across relevant numeric columns.
- Any country that was not present in life_exp_clean was removed from the WHO dataset before merging.
- A merge operation replaced old values with WHO data, ensuring alignment across datasets.

2.3 Under-5 Mortality

- The process for under-5 mortality was identical to the steps for infant mortality:
 - All causes of under-5 deaths were summed into a new column.
 - Countries not found in life_exp_clean were dropped.
 - A merge operation replaced old values with WHO data to ensure alignment. The final dataset contained cleaned and integrated under-5 mortality data.

Key Takeaways:

- WHO data provided critical mortality statistics, integrated into life_exp_clean.
- Country names were already aligned, simplifying merging.
- Infant and under-5 mortality data required aggregation before merging.
- A left join ensured that all countries in the main dataset were preserved.

3. Our World in Data (OWID) Datasets? 3.1 BMI (Male & Female)

- Country names in OWID were checked against life_exp_clean.
- Similar to the World Bank process, extractOne was used to match country names.
- Manual corrections included:
 - “Lao People’s Democratic Republic” → “Laos”

- “Republic of Korea” → “South Korea”
- “The former Yugoslav Republic of Macedonia” → “North Macedonia”
- “United Kingdom of Great Britain and Northern Ireland” → “United Kingdom”
- The `bmi_male` and `bmi_female` columns were updated using cleaned OVID data.
- The `Code` column was dropped, and the dataset was sorted by country and year.

Final Adjustments and Verifications

- Sorting and Indexing: All datasets were sorted by country and year.
- Data Integrity Checks: The number of unique countries was verified to be 176.
-

2.2 Standardization Across Sources: All variables were aligned with `life_exp_clean`, ensuring a unified and cleaned dataset for further analysis.

2.3 2.2 Filtering and Feature Selection

This section describes the process of filtering and selecting relevant features from the cleaned dataset to create a structured and standardized dataset for descriptive, exploratory, and inferential data analysis. This dataset forms the foundation for subsequent visualizations and statistical modeling.

2.3.1 1. Retained Variables

To ensure the dataset remains focused on key indicators relevant to life expectancy and economic conditions, the following variables were retained:

- `country` – Name of the country (176 unique countries)
- `year` – Observation year (2000 to 2015)
- `life_expectancy` – Probability of life expectancy at birth
- `adult_mortality` – Adult deaths per 1,000 adult population
- `infant_deaths` – Infant deaths per 1,000 infant population
- `under_five_deaths` – Under-five deaths per 1,000 under-five population
- `bmi_male` – Average BMI of the entire male population
- `bmi_female` – Average BMI of the entire female population
- `health_expenditure_percentage` – Expenditure on health as a percentage of Gross Domestic Product (GDP) per capita
- `total_expenditure` – General government expenditure on health as a percentage of total government expenditure
- `population` – Total population of the country
- `gdp` – Gross Domestic Product (GDP) per capita
- `gni` – Gross National Income (GNI) per capita

By filtering out unnecessary variables, the dataset remains streamlined for efficient analysis.

2.3.2 2. Filtering the Columns

To extract only the relevant variables, the following filtering process was applied:

```
life_exp_clean_2 = life_exp_clean.copy()
```



```
life_exp_clean_2 = life_exp_clean_2[['country', 'year', 'life_expectancy', 'adult_mortality',
                                     'bmi_female', 'health_expenditure_percentage', 'total_exper
```

```
life_exp_clean_2.rename(columns={'gdp': 'gdp_per_capita'}, inplace=True)
```

This ensures that the final dataset includes only the variables necessary for analysis while renaming GDP to gdp_per_capita for clarity.

2.3.3 3. Converting Variables to Correct Data Types

To maintain consistency and accuracy in numerical computations, data types were adjusted as follows:

```
life_exp_clean_2[['year', 'population']] = life_exp_clean_2[['year', 'population']].astype('Int64')
```

```
float_columns = life_exp_clean_2.select_dtypes(include='Float64').columns
```

```
life_exp_clean_2[float_columns] = life_exp_clean_2[float_columns].round(2)
```

```
life_exp_clean_2['country'] = life_exp_clean_2['country'].astype('string')
```

```
life_exp_clean_2['year'] = pd.to_datetime(life_exp_clean_2['year'], format='%Y').dt.year
```

- The year and population columns were converted to integers (Int64) to reflect their discrete nature.
- All floating-point numerical values were rounded to two decimal places to enhance readability.
- The country column was converted to a string type for categorical operations.
- The year column was briefly converted to a datetime format and then back to integer format to enable potential future time-series analysis.

These transformations ensure that the dataset is structured optimally for analysis, minimizing inconsistencies and formatting issues.

Final Adjustments and Verification

- Sorting and Indexing: The dataset was sorted by country and year to maintain chronological consistency.
- Data Integrity Checks: The total number of unique countries was verified to remain at 176.
- Standardization: All columns align with the predefined schema, ensuring uniformity across the dataset.

This structured approach results in a cleaned and well-formatted dataset, ready for further analysis and visualization.

2.4 2.3 Enriching the Data

This section describes the processes used to enrich the dataset by adding regional and sub-regional classifications and assigning income classes based on World Bank classifications. These enhancements provide additional layers of analysis, enabling a more comprehensive understanding of how geographic and economic factors relate to life expectancy.

2.4.1 1. Adding Regional and Sub-Regional Classifications

To enhance the dataset with regional classifications, all 176 countries included in the analysis were manually categorized into their respective regions and sub-regions. This classification was necessary to facilitate regional comparisons and better contextualize variations in life expectancy and other key indicators.

Manual Categorization Process

- Country Listings: A complete list of 176 countries was reviewed, ensuring coverage across all geographic regions.
- Dictionaries for Sorting: Manually created dictionaries (`countries_by_region`) were developed to assign each country to its appropriate sub-region.
- DataFrame Creation: Separate sub-regional DataFrames were generated to store country classifications.
- Concatenation into Regional DataFrames:
 - Sub-regional DataFrames were concatenated using the `pd.concat()` function.
 - The final regional DataFrames were created using:
`pd.concat(sub-regions).reset_index(drop=True).sort_values(by=['country'], ascending=True)`
- Exporting Cleaned Datasets:
 - The newly categorized data was exported using `.to_csv()` to create structured CSV files.
 - These files were saved in the project directory: `Datasets/Cleaned_datasets/World_regions`.

Integration into the Main Dataset

Two new categorical variables, `region` and `sub-region`, were added to the `life_exp_clean_2` DataFrame using a multi-loop classification system:

- A dictionary (`sub_regions_dict`) was created to map sub-regions to their corresponding regions.
- An iterative loop assigned each country to its respective region and sub-region.
- The classification was then stored in lists and appended to the main dataset.

The following code snippet illustrates the classification process:

```
# Generate lists of sub-regions
sub_region_keys = list(countries_by_region.keys())
sub_regions_dict = {
    'Asian region': sub_region_keys[0:5],
    'European region': sub_region_keys[5:9],
    'African region': sub_region_keys[9:14],
    'American region': sub_region_keys[14:18],
    'Oceania region': sub_region_keys[18:22]
}

# Assign region and sub-region classifications
region_classification_list = []
sub_region_classification_list = []
```

```

for country in list(life_exp_clean_2['country']):
    for sub_region in countries_by_region:
        if country in countries_by_region[sub_region]:
            for regions in sub_regions_dict:
                if sub_region in sub_regions_dict[regions]:
                    region_classification_list.append(regions)
                    sub_region_classification_list.append(sub_region)

# Add region and sub-region classifications as string-type columns
life_exp_clean_2['region'] = region_classification_list.astype('string')
life_exp_clean_2['sub_region'] = sub_region_classification_list.astype('string')

```

The inclusion of regional and sub-regional classifications allows for structured comparative analysis across different geographic areas.

2.4.2 2. Assigning Income Classes Based on World Bank Classifications

To further enrich the dataset, each country was assigned an income class based on the World Bank's classification thresholds. This classification aids in examining how economic factors impact life expectancy and related health indicators.

Income Classification Thresholds

The following World Bank income thresholds (2015 values) were applied:

- Low Income: GNI per capita < 1,045 USD
- Lower-Middle Income: GNI per capita < 4,125 USD
- Upper-Middle Income: GNI per capita < 12,735 USD
- High Income: GNI per capita > 12,735 USD

Automated Income Classification Process

To categorize each country, the following approach was implemented:

1. The income class was assigned based on each country's Gross National Income (GNI) per capita in 2015.
2. The classification was determined using a lambda function within a apply() method.
3. Missing values were forward-filled (ffill()) to ensure consistency.

The following Python code was used:

```

life_exp_clean_2['income_class'] = life_exp_clean_2['gni'][life_exp_clean_2['year'] == 2015].apply(
    lambda x: 'Low income' if x <= 1045 else
              'Lower-middle income' if x <= 4125 else
              'Upper-middle income' if x <= 12735 else
              'High income'
)

# Forward-fill missing values
life_exp_clean_2['income_class'] = life_exp_clean_2['income_class'].ffill()

```

This process ensures that all 176 countries are correctly classified according to regional, sub-regional, and income groupings, enriching the dataset for advanced analysis and visualization.

2.5 2.4 Handling Missing Data

2.5.1 2.4.1 Inspection

To ensure data integrity and reliability in the analysis, a thorough inspection of missing values was conducted. The dataset was examined to identify missing values across all variables, including added categorical attributes such as region, sub-region, and income class.

Identification of Missing Data

A dictionary was generated to display all variables along with the corresponding count of missing entries. Additionally, a Missing Value Percentage Table was created to visualize the proportion of missing data per variable, highlighting areas that required imputation, extrapolation, or removal.

A key step in this inspection was identifying specific countries with missing values and quantifying these gaps. This was achieved by generating a dictionary of missing values per country, which facilitated a structured approach to addressing data gaps systematically.

Patterns in Missing Data

To categorize missing data trends, three distinct conditions were identified:

1. Countries missing values only for earlier years – These cases suggest that data collection for these countries commenced after a certain period.
2. Countries missing values only for later years – These cases indicate an abrupt cessation of data recording, possibly due to political instability, economic downturns, or shifts in reporting standards.
3. Countries missing all values – These cases highlight a complete lack of recorded data for specific variables, necessitating alternative handling strategies such as exclusion or alternative estimations.

The missing data distribution was further analyzed in the context of three key variables requiring imputation:

- Health Expenditure Percentage and Total Expenditure (Government Health Expenditure): Missing values were recorded for the same set of countries and years, namely Afghanistan, Iraq, Montenegro, Timor-Leste, Zimbabwe, Libya, and the Syrian Arab Republic.
- Gross National Income (GNI per capita): Missing values were primarily observed in earlier years for Afghanistan, Djibouti, and Nigeria.

Challenges in Data Imputation

- The dataset contained only 16 yearly observations per country per variable, limiting the training capability of the imputer.
- Due to insufficient data points, multiple imputation could only estimate a single fitting value rather than a range of plausible values for each missing entry.
- Economic fluctuations further complicated imputation, as drastic changes in macroeconomic variables were difficult to account for with limited data.
- In practice, this meant that for multiple imputation techniques, a single fitting value was often used to impute all missing data for a variable, reducing the robustness of the imputation model.

Testing Multiple Imputation Approaches

Several imputation strategies were explored to determine the most effective method:

- Imputation using regional-level aggregate data
- Imputation using sub-regional aggregate data
- Imputation using all non-missing data

Correlation Analysis and Insights

A seaborn heatmap of correlation matrices was employed to investigate potential relationships between missing variables and other recorded variables. The key findings were as follows:

1. **Health Expenditure Percentage:** Displayed weak correlations with other variables, limiting the effectiveness of predictive imputation. The highest correlations observed were:
 - Life Expectancy: 0.36
 - Adult Mortality: 0.22
 - Infant Deaths: 0.36
 - Under-5 Deaths: 0.28
 - Male BMI: 0.36
 - Female BMI: 0.15
 - GDP per Capita: 0.35
 - GNI per Capita: 0.41
2. **Total Expenditure (Government Health Expenditure):** Exhibited stronger correlations, suggesting potential for imputation based on related variables. Notable correlations included:
 - Life Expectancy: 0.55
 - Adult Mortality: 0.40
 - Infant Deaths: 0.58
 - Under-5 Deaths: 0.46
 - Male BMI: 0.47
 - Female BMI: 0.28
 - GDP per Capita: 0.41
 - GNI per Capita: 0.41

Summary of Missing Data Handling Methods

Based on the above analysis, three primary techniques were employed to handle missing data:

1. Imputation: Applied to variables where sufficient correlation with other variables enabled reasonable estimates, such as GNI using GDP per capita trends.
2. Extrapolation: Used where missing data followed an identifiable temporal trend, allowing for reasonable projection of missing values.
3. Removal: Countries with missing values across all years for a variable were excluded from the dataset, as no meaningful estimation could be performed.

These methodologies were applied accordingly in the following sections to address missing values for Health Expenditure Percentage, Total Expenditure (Government Health Expenditure), and GNI.

2.5.2 2.4.2 Creating Multiple Imputation Class

Introduction

Missing data poses a significant challenge in any analysis, particularly when working with economic and health indicators where data collection may not be consistently available across all regions and years. To address the missing values in the `health_expenditure_percentage` and `total_expenditure` variables, a Multiple Imputation Class was developed. This class leverages an advanced imputation technique by identifying countries with similar economic and health characteristics to serve as reference data for missing entries.

Identifying Missing Data Patterns

A detailed analysis of missing data patterns revealed that for `health_expenditure_percentage` and `total_expenditure`, five countries—Afghanistan, Iraq, Montenegro, Timor-Leste, and Zimbabwe—had missing data primarily in the earlier years (2000 onward). Given that data collection improved in later years, it was inferred that the missing data was likely missing at random (MAR) rather than completely missing without a pattern. This assumption supported the approach of using similar countries with complete data for imputation.

Exploring Different Imputation Approaches

Initially, multiple strategies were tested to train the `IterativeImputer`:

1. Using regional data (aggregating data within broader regions).
2. Using sub-regional data (a more localized approach).
3. Using the entire dataset for training the imputer.

None of these methods produced reliable results due to the limited number of observations per country (only 16 yearly data points). This limitation meant that a single fitted value was often used across all missing entries, reducing the accuracy and robustness of the imputation.

Final Imputation Strategy

The optimal method involved finding countries with similar characteristics (excluding the missing variable) and then training the imputer on these selected countries. This approach allowed for a more contextually relevant imputation by ensuring that missing values were filled using countries with comparable economic and health profiles.

The imputation class, `CountryImputer`, follows these steps:

1. Identify similar countries based on shared economic and health characteristics.
2. Train an imputer using data from these selected countries, excluding the variable being imputed.
3. Apply imputation to fill missing values for the target country.

This methodology accounted for structural similarities between countries rather than relying on arbitrary regional groupings.

Class Implementation and Functionality

Step 1: Finding Similar Countries

The `find_similar_countries` method identifies relevant reference countries using a tolerance-based filtering approach:

- A set of core variables (life_expectancy, adult_mortality, infant_deaths, under_five_deaths, bmi_male, bmi_female, gdp_per_capita, and gni) are considered.
- The method calculates maximum and minimum values for each variable within the target country.
- A tolerance margin is applied to these values to create a range within which other countries should fall.
- Countries whose variable values fall within this tolerance range are selected as reference data.

This method ensures that only countries with comparable economic and health characteristics are used in the imputation process.

Step 2: Imputing Missing Data

The `impute_country` method applies Iterative Imputation to the selected reference data:

- The data is first normalized (e.g., GDP per capita and life expectancy are scaled for consistency).
- Certain variables (e.g., adult_mortality, infant_deaths) are inverted since lower values correlate with higher economic and health indicators.
- The `IterativeImputer` is trained using the selected reference countries, excluding the missing variable.
- The trained imputer is used to predict missing values for the target country.

Addressing Economic Fluctuations

One of the key challenges encountered was the economic fluctuations over time, which could not always be accounted for with the limited available data. Since imputation relies on patterns within existing data, unpredictable shifts in economic trends (such as political instability or sudden growth spurts) could reduce accuracy. Despite this limitation, the chosen method produced more reliable estimates than other tested approaches.

Example Usage of the Imputation Class

To demonstrate the application of the `CountryImputer` class, consider the following example where Afghanistan's `health_expenditure_percentage` is imputed:

```
finder = CountryImputer(life_exp_clean_2, 'Afghanistan', 'health_expenditure_percentage')
considered_countries = finder.find_similar_countries(10, ['bmi_male', 'total_expenditure', 'gni'])
hep_multiple_imp = finder.impute_country(considered_countries)
```

In this case:

- A 10% tolerance was applied to find similar countries.
- Certain variables (bmi_male, total_expenditure, gni) were excluded to focus on economic and mortality-related indicators.
- The resulting imputation was applied only to Afghanistan for the missing years.

Outputs and Verification

Throughout the imputation process, several diagnostic outputs were generated to ensure transparency:

1. Max and Min values per variable for the target country.
2. Max and Min values with applied tolerance to identify similar countries.
3. List of selected reference countries used for imputation.

4. Years with missing data to track imputed entries.
5. Final imputed dataset preview to validate the results.

Key notes

The development of this Multiple Imputation Class provides a structured and adaptable approach to handling missing data in economic and health datasets. By leveraging similarities between countries, this method ensures that missing values are imputed in a contextually relevant manner, improving the integrity and completeness of the dataset. While economic fluctuations remain a challenge, the approach provides a significantly better alternative to traditional imputation methods, ensuring that meaningful analyses can be conducted without the bias introduced by arbitrary data removal or simplistic imputation techniques.

2.5.3 2.4.3 Filling the missing values:

2.4.3.1 Health Expenditure Percentage Identifying Missing Values:

During the inspection of missing values in the `health_expenditure_percentage` variable, the following countries were found to have incomplete data:

```
{'Afghanistan': 2, 'Iraq': 3, 'Libya': 4, 'Montenegro': 11, 'Somalia': 16, 'Syrian Arab Republ.
```

To address these missing values, a multiple imputation strategy was adopted using a custom-built `CountryImputer` class, which applies `IterativeImputer` from Scikit-learn under the hood. This allowed for imputation based on similar countries and similar features, maintaining realistic values consistent with each country's economic and health profile.

Countries Imputed Using `CountryImputer`

1. Afghanistan

- **Missing Years:** 2000, 2001
- **Similar countries:** Guinea-Bissau, Liberia, Malawi, Rwanda
- **Excluded Features:** `bmi_male`, `total_expenditure`, `gni`
- **Imputed Values:** {2000: 10.11, 2001: 10.11}

2. Iraq

- **Missing Years:** 2000, 2001, 2002
- **Similar countries:** Azerbaijan, Egypt, Kazakhstan, Mongolia, Morocco, etc.
- **Excluded Features:** `bmi_male`, `total_expenditure`, `gni`
- **Imputed Values:** {2000: 3.16, 2001: 3.16, 2002: 3.16}

3. Montenegro

- **Missing Years:** 2000–2010 (11 years)
- **Similar countries:** Bosnia and Herzegovina, Croatia, Serbia, Slovakia, etc.
- **Excluded Features:** `bmi_male`, `total_expenditure`, `gni`
- **Imputed Values:** All missing years were imputed with 7.23

4. Timor-Leste

- **Missing Years:** 2000–2002
- **Similar countries:** Cambodia, India, Indonesia, Myanmar, etc.

- **Excluded Features:** bmi_male, total_expenditure, gni
- **Imputed Values:** {2000: 4.41, 2001: 4.41, 2002: 4.41}

5. Zimbabwe

- **Missing Years:** 2000–2009 (10 years)
- **Similar countries:** Malawi, Mozambique, Rwanda, Uganda, Zambia
- **Excluded Features:** bmi_male, total_expenditure, gni
- **Imputed Values:** All missing years were imputed with 5.87

Each imputation followed the same methodology:

1. Identify similar countries using selected features correlated with healthcare investment.
2. Apply iterative imputation constrained by max-min bounds and a tolerance range derived from existing values.
3. Evaluate the resulting values to ensure consistency with country-specific trends.

Imputation Using Extrapolation: Libya and Syrian Arab Republic For countries where health expenditure percentage data was missing after a few years of available records, extrapolation techniques were used to fill in the gaps. Two different regression approaches were applied depending on the trend characteristics observed.

1. Libya: Polynomial Regression Imputation

- **Model Evaluation:**
 - **R² Score:** 0.0816
 - **Mean Squared Error (MSE):** 0.538
 - The low R² score and high MSE suggest that a linear model is a poor fit, indicating a non-linear trend.
- **Trend Observation:**
 - A clear upward trend in health expenditure percentage was observed for all countries in the Northern African sub-region.
 - Despite potential political reasons for missing data, a second-degree polynomial regression was used to extrapolate values.
- **Implementation:** `python poly_model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression()) poly_model.fit(recorded_years, recorded_hep_values) poly_predicted_values = poly_model.predict(missing_years)`
- **Imputed Values:** `python {2012: 4.03, 2013: 4.49, 2014: 5.03, 2015: 5.64}`
 - These values were used to fill in the missing years from 2012 onward.

2. Syrian Arab Republic: Linear Regression Imputation

- **Model Evaluation:**
 - R² Score: 0.847
 - Mean Squared Error (MSE): 0.087
 - These metrics indicate that linear regression is a strong fit for this country's data.
- **Trend Observation:**
 - A linear relationship was evident in the scatterplot of recorded data.
- **Implementation:** `python model = LinearRegression() model.fit(recorded_years, recorded_hep_values) missing_hep_values_predict`

- = model.predict(missing_years)
- Imputed Values:** python {2013: 2.64, 2014: 2.45, 2015: 2.27} The linear trend suggests that the missing values likely follow the recorded pattern, possibly withheld due to political unrest.

Dropping Somalia from the Dataset During the data cleaning and imputation process, Somalia was removed from the dataset due to a complete absence of health expenditure percentage data across all years (2000–2015). Unlike other countries with partial missing data, Somalia lacked any available records, making it impossible to apply standard imputation techniques such as interpolation or extrapolation.

Further research into Somalia’s economic and healthcare reporting context confirmed that this was not an oversight. Somalia stands out as an exception among African nations, notably not adhering to the Abuja Declaration adopted by the African Union, which recommends allocating at least 15% of national budgets to health. Instead, estimates suggest Somalia’s health expenditure is consistently below 2%, with no reliable figures officially reported by the World Health Organization (WHO) during the study period.

Given the absence of comparable data and the unique political and economic situation of the country, retaining Somalia in the dataset would introduce substantial bias and uncertainty. Therefore, Somalia was justifiably excluded from the project dataset to maintain the overall data integrity and focus on countries with meaningful and verifiable health expenditure data.

2.4.3.2 Total Expenditure (Government Health Expenditure) Identifying Missing Values

Upon inspection of the total_expenditure column—representing government health expenditure as a percentage of total government spending—a total of eight countries were found to contain missing values. The missing value counts per country were:

```
{'Afghanistan': 2, 'Iraq': 3, 'Libya': 4, 'Montenegro': 11, 'Somalia': 16, 'Syrian Arab Republ.
```

The missing values occurred across two distinct patterns: countries with missing data in the earlier years (e.g., 2000–2005), and those with data gaps in the later years (e.g., post-2011). First we’ll focus on countries with missing values primarily in the earlier years: Afghanistan, Iraq, Libya, Montenegro, Timor-Leste, and Zimbabwe.

Countries Imputed Using CountryImputer

1. Afghanistan

- **Missing Years:** 2000, 2001
- **Excluded Features:** ['bmi_male', 'bmi_female', 'gni']
- **Similar Countries:** Guinea-Bissau, Liberia, Malawi, Rwanda
- **Imputed Values:** {2000: 2.01, 2001: 2.01}

2. Iraq

- **Missing Years:** 2000–2002
- **Excluded Features:** ['bmi_male', 'gdp_per_capita', 'gni']
- **Similar Countries:** Azerbaijan, Egypt, Morocco, Tajikistan
- **Imputed Values:** {2000: 2.82, 2001: 2.82, 2002: 2.82}

3. Montenegro

- **Missing Years:** 2000–2010 (11 years total)
- **Excluded Features:** ['bmi_male', 'gni']
- **Similar Countries:** Bosnia and Herzegovina, Bulgaria, Chile, Croatia, Cuba, Czechia, Estonia, Hungary, Lebanon, Maldives, Montenegro, Poland, Serbia, Slovakia, The former Yugoslav republic of Macedonia
- **Imputed Values:** 2000–2010: 10.81% (consistent across years)

4. Timor-Leste

- **Missing Years:** 2000–2002
- **Excluded Features:** ['bmi_male', 'gni']
- **Similar Countries:** Indonesia, Nepal, Yemen, among others
- **Imputed Values:** 2000–2002: 2.86%

5. Zimbabwe

- **Missing Years:** 2000–2009
- **Excluded Features:** ['bmi_male', 'gdp_per_capita', 'gni']
- **Similar Countries:** Malawi, Rwanda, Tanzania, Uganda
- **Imputed Values:** 2000–2009: 8.18%

Imputation Using Extrapolation: Libya and Syrian Arab Republic In the dataset, both Libya and the Syrian Arab Republic have missing values for government health expenditure (total_expenditure) from 2012 onward and 2013 onward, respectively. Unlike countries where data is missing in earlier years due to late adoption of data collection, the absence of values in these later years suggests a different mechanism — Not Missing at Random (NMAR). It is reasonable to assume that political instability and civil conflict, which escalated during the missing periods, influenced the availability and publication of official health expenditure data.

Given this context, extrapolation was used to estimate the missing values using either linear or polynomial regression, depending on the observed data trend. Visual inspection through scatter plots was conducted to determine the appropriate method for each country.

1. Libya: Polynomial Regression Imputation

- **Model Evaluation:**
 - R^2 Score: 0.308
 - Mean Squared Error (MSE): 0.817
 - The low R^2 score indicates a weak linear relationship — the linear model explains only about 30.8% of the variance in the observed data. The relatively high MSE further supports that a linear model is not a good fit for Libya’s trend in government health expenditure.
- **Trend Observation:**
 - The scatter plot of Libya’s recorded total_expenditure data showed a non-linear pattern, inconsistent with a straight-line trend.
 - While Libya’s values fluctuated, neighboring North African countries displayed a clearer upward trend. This regional context, along with Libya’s earlier trajectory, supported the use of polynomial regression to better capture the curved progression.

- **Implementation:** `python poly_model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression()) poly_model.fit(recorded_years, recorded_hep_values) poly_predict_known_values = poly_model.predict(recorded_years) poly_predicted_values = poly_model.predict(missing_years) rounded_poly_predicted_values = np.round(poly_predicted_values, 2)`
- **Imputed Values:** `python {2012: 5.07, 2013: 5.23, 2014: 5.44, 2015: 5.7}`

2. Syrian Arab Republic: Linear Regression Imputation

- **Model Evaluation:**
 - R^2 Score: 0.416
 - Mean Squared Error (MSE): 0.733
 - The R^2 score indicates a moderate fit — the linear model explains approximately 41.6% of the variance. The MSE is slightly lower than Libya's, suggesting a more stable and consistent trend. Overall, the linear model was reasonably well-suited for Syria's expenditure trend.
- **Trend Observation:**
 - The scatter plot revealed a clear and consistent downward linear trend in total expenditure before 2013.
 - This supported the use of linear regression to estimate missing values from 2013 to 2015, in line with the observed trajectory.
- **Implementation:** `python model = LinearRegression() model.fit(recorded_years, recorded_hep_values) missing_te_values_predict = model.predict(missing_years)`
- **Imputed Values:** `python {2013: 4.87, 2014: 4.68, 2015: 4.49}` The linear trend suggests that the missing values likely follow the recorded pattern, possibly withheld due to political unrest.

2.4.3.3 GNI per Capita Overview

During the inspection of the gni (Gross National Income per capita) column, missing values were identified in three countries. The following dictionary summarizes the number of missing values per country:

```
{'Afghanistan': 2, 'Djibouti': 15, 'Nigeria': 8}
```

Given that GNI per capita closely mirrors the trends of GDP per capita, both in economic reporting and statistical progression, it was deemed reasonable to use GDP per capita as a predictor in imputing the missing GNI values. This assumption allowed the use of Iterative Imputation methods that model missing data based on multivariate relations, specifically between GDP per capita and GNI per capita.

Imputation Approach

For all three countries, the IterativeImputer from Scikit-learn was used. This approach models each feature with missing values as a function of other features and iteratively refines the imputed values. Since the relationship between GDP per capita and GNI per capita tends to be linear and stable over time, it served as an appropriate foundation for imputation.

1. Afghanistan

- **Imputation Logic:** Afghanistan had sufficient non-missing data points to train its imputer using only its own records.
- **Implementation:**
 - **Selected columns:** ['gdp_per_capita', 'gni']
 - Fit and transformed values using IterativeImputer
 - Rounded results to one decimal place
 - Replaced missing gni values in Afghanistan's records using the imputed output
- **Imputed Values:** python {2000: 131.8, 2001: 169.4}
- **Justification:** Since Afghanistan's dataset had only two missing values and a good quantity of existing records, the internal trend between GDP and GNI was expected to yield accurate predictions without requiring external regional data.

2. Djibouti

- **Imputation Logic:** Djibouti had 15 missing GNI values, which made internal training less reliable. Instead, the imputer was trained using the complete GDP and GNI data of all countries within the East African sub-region, to which Djibouti belongs.
- **Implementation:**
 - Subset of training data: East African countries with complete gdp_per_capita and gni
 - Trained IterativeImputer on this regional data
 - Transformed missing values for Djibouti
 - Replaced missing gni values using the second column of the imputer output
- **Imputed Values:** python {2000: 706.24, 2001: 710.99, 2002: 712.25, 2003: 733.81, 2004: 774.96, 2005: 812.68, 2006: 865.83, 2007: 935.69, 2008: 1082.37, 2009: 1113.92, 2010: 1175.42, 2011: 1267.24, 2012: 1359.86, 2013: 2021.37, 2014: 2153.82}
- **Justification:** Regional socioeconomic factors are often shared within geographical groupings. Thus, imputing based on sub-regional data provided a stronger model than relying solely on Djibouti's limited historical data.

3. Nigeria

- **Imputation Logic:** Nigeria had 8 missing values. Similar to Djibouti, the imputer was trained using complete GDP and GNI values from the West African sub-region, which reflects Nigeria's regional economic context.
- **Implementation:**
 - Subset of training data: West African countries with no missing gdp_per_capita or gni
 - Trained and applied IterativeImputer to predict Nigeria's missing gni values
 - Replaced missing values in Nigeria's dataset accordingly
- **Imputed Values:** python {2000: 537.7, 2001: 536.2, 2002: 695.6, 2003: 744.9, 2004: 935.7, 2005: 1174.4, 2006: 1546.5, 2007: 1754.2}
- **Justification:** Although Nigeria has a large economy, regional context still improves imputation by capturing the economic behavior of peer nations with shared development patterns.

2.6 2.5 Data Type Adjustment

To optimize both memory efficiency and semantic clarity in the dataset, appropriate data type conversions were performed. Specifically, categorical variables were explicitly cast to the category data type in pandas. This step is essential for large datasets as it reduces memory usage and improves performance in operations such as grouping, sorting, and filtering. Additionally, assigning ordered

categories can help preserve logical hierarchies in later visualizations and statistical modeling.

2.6.1 2.5.1 Income Class

The `income_class` column represents a natural ordinal scale ranging from low to high-income categories. This variable was explicitly converted into an ordered categorical type, following the World Bank classification:

```
['Low income', 'Lower-middle income', 'Upper-middle income', 'High income']
```

By setting the `ordered=True` parameter, the column retains its inherent order, which is useful for ordered visualizations and analysis (e.g., boxplots segmented by income class).

2.6.2 2.5.2 Region

The `region` column, while not ordinal, represents discrete geopolitical zones (e.g., Asia, Africa, Europe). This was converted to an unordered categorical type, ensuring efficient grouping and summarization without implying any hierarchy.

2.6.3 2.5.3 Sub-region

Similar to `region`, the `sub_region` column was also cast to a categorical type. This preserves its use for stratified analysis while benefiting from optimized storage.

By converting these columns, the dataset is not only lighter in memory but is also more semantically aligned for downstream analysis and plotting tasks, especially in libraries such as `seaborn` and `matplotlib`.

2.6.4 2.5.4 Display dataset information

The cleaned Life Expectancy dataset contains 2,800 entries (rows) and 16 columns, all of which have no missing values, indicating that the dataset is fully preprocessed and analysis-ready.

```

RangeIndex: 2800 entries, 0 to 2799
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype  
---  -
0   country                               2800 non-null   string  
1   year                                  2800 non-null   int32   
2   life_expectancy                       2800 non-null   float64  
3   adult_mortality                       2800 non-null   float64  
4   infant_deaths                         2800 non-null   float64  
5   under_five_deaths                     2800 non-null   float64  
6   bmi_male                              2800 non-null   float64  
7   bmi_female                            2800 non-null   float64  
8   health_expenditure_percentage         2800 non-null   float64  
9   total_expenditure                     2800 non-null   float64  
10  population                             2800 non-null   Int64   
11  gdp_per_capita                         2800 non-null   float64  
12  gni                                    2800 non-null   float64  
13  region                                2800 non-null   category
14  sub_region                            2800 non-null   category
15  income_class                           2800 non-null   category
dtypes: Int64(1), category(3), float64(10), int32(1), string(1)
memory usage: 285.6 KB

```

Data Types Overview

- Numeric Columns:
 - int32: 1 column (year)
 - int64: 1 column (population)
 - float64: 10 columns (life_expectancy, adult_mortality, infant_deaths, under_five_deaths, bmi_male, bmi_female, health_expenditure_percentage, total_expenditure, gdp_per_capita, gni)
- Categorical Columns:
 - category: 3 columns (region, sub_region, income_class)
- String Columns:
 - string: 1 column (country)

Memory Usage

The entire dataset uses approximately 285.6 KB of memory, which is notably efficient given the data volume. This optimization was aided by converting relevant columns (e.g., regions and income classes) to the category data type.

Column Completeness

All columns have 2,800 non-null entries, confirming successful handling of previously missing values through appropriate imputation and cleaning strategies.

2.7 2.6 Exporting Cleaned Dataset to CSV

With all cleaning, imputation, and formatting processes completed, the fully processed DataFrame (life_exp_clean_2) was saved to a CSV file for future access and reproducibility:

```
life_exp_clean_2.to_csv("../Datasets/Cleaned_datasets/Life_expectancy_cleaned_1.csv")
```

The file was exported to the `Cleaned_datasets` directory under the main `Datasets` folder. This clean, analysis-ready file serves as the finalized version of the dataset for use in visualization, modeling, and reporting tasks in subsequent phases of the project. Saving this cleaned version ensures that all data wrangling efforts are preserved and provides a clear separation between raw and prepared data sources.